Artificial Neural Network (CS642)

Lecture 06 - Optimization Methods and Regularization

Dr. Abdul Basit

Assistant Professor - CSIM
University of Balochistan

September 25, 2024

Reference

- Material taken from Chapter 9 of " Deep Learning for Computer Vision with Python" by "Dr. Adrian Rosebrock."

Contents

Optimization methods and regularization

Background

- Optimization algorithms are the engines that power neural networks and enable them to learn patterns from data.
- We have learned that obtaining a high accuracy classifier is dependent on finding a set of weights $\mathtt{W}$ and $\boldsymbol{b}$ such that our data points are correctly classified.

- How do we go about finding and obtaining a weight matrix `W` and bias vector $\boldsymbol{b}$ that obtains high classification accuracy?
- Do we randomly initialize them, evaluate, and repeat over and over again, hoping that at some point we land on a set of parameters that obtains reasonable classification?
- But there are number parameters to train and it will take longer time.

- Instead of relying on pure randomness, we need to define an optimization algorithm that allows us to literally improve `W` and $\boldsymbol{b}$.
- We use gradient descent commonly used optimization algorithm in neural network. In each case, iteratively evaluate the parameters, compute your loss, then take a small step in the direction that will minimize your loss.

Contents

Gradient descent

- The gradient descent algorithm has two primary flavors:
  - The standard vanilla implementation.
  - The optimized stochastic version that is more commonly used.

Contents

Gradient descent

The Loss Landscape and Optimization Surface

- The gradient descent method is an iterative optimization algorithm that operates over a loss landscape (also called an optimization surface).

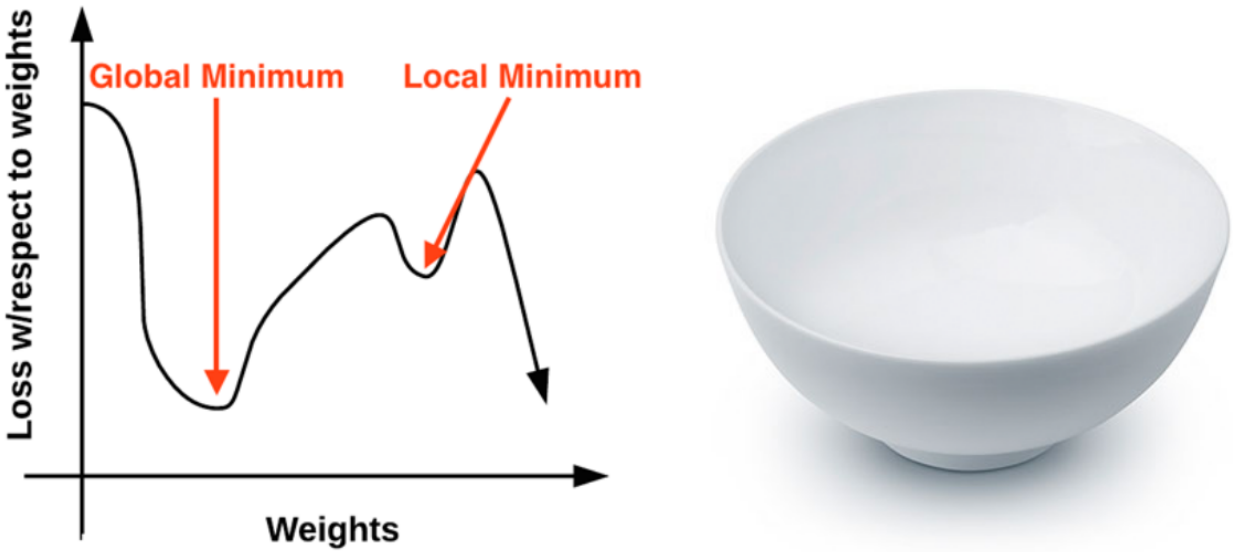

Figure: **Left**: The "naive loss" visualized as a 2D plot. **Right:** A more realistic loss landscape can be visualized as a bowl that exists in multiple dimensions. Our goal is to apply gradient descent to navigate to the bottom of this bowl (where there is low loss).

Gradient descent

The Loss Landscape and Optimization Surface

- Each peak is a local maximum that represents very high regions of loss – the local maximum with the largest loss across the entire loss landscape is the global maximum. Similarly, we also have local minimum which represents many small regions of loss.
- The local minimum with the smallest loss across the loss landscape is our global minimum. In an ideal world, we would like to find this global minimum, ensuring our parameters take on the most optimal possible values.
- So that raises the question: *"If we want to reach a global minimum, why not just directly jump to it? It's clearly visible on the plot?"*

Gradient descent

The Loss Landscape and Optimization Surface

- Actually the loss landscape is invisible to us. We do not know what it looks like. So, we would have to navigate our way to a loss minimum without accidentally climbing to the top of a local maximum.
- Let's look at a different visualization of the loss landscape that does a better job depicting the problem. Here we have a bowl, similar to the one you may eat cereal or soup out of See Figure above right.

Gradient descent

The Loss Landscape and Optimization Surface

- The surface of our bowl is the loss landscape, which is a plot of the loss function. The difference between our loss landscape and our cereal bowl is that your cereal bowl only exists in three dimensions, while your loss landscape exists in many dimension, perhaps tens, hundreds, or thousands of dimensions.
- Each position along the surface of the bowl corresponds to a particular loss value given a set of parameters $\mathtt{W}$ and $\boldsymbol{b}$. Our goal is to try different values of $\mathtt{W}$ and $\boldsymbol{b}$, evaluate their loss, and then take a step towards more optimal values that (ideally) have lower loss.

Contents

Gradient descent

The "Gradient" in Gradient Descent

- To make our explanation of gradient descent a little more intuitive, let's pretend that we have a robot. When performing gradient descent, we randomly drop the robot somewhere on our loss landscape.

Figure: **Left**: Our robot, Chad. **Right**: It's Chad's job to navigate our loss landscape and descend to the bottom of the basin. Unfortunately, the only sensor Chad can use to control his navigation is a special function, called a *loss function*, L. This function must guide him to an area of lower loss.

- It's now the robot's job to navigate to the bottom of the basin which is the minimum loss. All the robot has to do is orient himself such that he's facing downhill and ride the slope until he reaches the bottom of the bowl.
- But Chad is only able to compute his relative position using parameters $\mathtt{W}$ and $\boldsymbol{b}$ on the loss landscape. He has absolutely no idea in which direction he should take a step to move himself closer to the bottom of the basin.

Gradient descent

The "Gradient" in Gradient Descent

- What is the robot to do? The answer is to apply gradient descent. All the robot needs to do is follow the slope of the gradient $\mathtt{W}$. We can compute the gradient $\mathtt{W}$ across all dimensions using the following equation:

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\mathtt{W}_{n+1} = \mathtt{W}_n - \alpha \frac{\partial f(\mathtt{w}, b)}{\partial \mathtt{w}}$$

- In > 1 dimensions, our gradient becomes a vector of partial derivatives. The problem with this equation is that:
  - It is an approximation to the gradient.
  - It is painfully slow.

Gradient descent

The "Gradient" in Gradient Descent

- In practice, we use the analytic gradient instead. The method is exact and fast, but extremely challenging to implement due to partial derivatives and multi-variable calculus.
- What gradient descent is: attempting to optimize our parameters for low loss and high classification accuracy via an iterative process of taking a step in the direction that minimizes loss.

Gradient descent

Treat It Like a Convex Problem (Even if It's Not)

- **We treat the loss landscape as a convex problem, even if it's not**. If some function F is convex, then all local minima are also global minima.
- The issue is that nearly all problems we apply neural networks and deep learning algorithms to are not neat, convex functions. Instead, inside this bowl we find spike-like peaks, where loss drops dramatically only to sharply rise again.

Gradient descent

Treat It Like a Convex Problem (Even if It's Not)

- Given the non-convex nature of our datasets, why do we apply gradient descent? The answer is simple: because it does a good enough job.
  "[An] optimization algorithm may not be guaranteed to arrive at even a local minimum in a reasonable amount of time, but it often finds a very low value of the [loss] function quickly enough to be useful."
- Even existence of this reality, we end up finding a region of low loss – this area may not even be a local minimum, but in practice, it turns out that this is good enough.

Contents

Gradient descent

The bias trick

- A method of combining our weight matrix $\mathtt{W}$ and bias vector $\boldsymbol{b}$ into a single parameter. Recall from our previous decisions that our scoring function is defined as:

$$f(x_i, \mathtt{W}, \boldsymbol{b}) = \mathtt{W}x_i + \boldsymbol{b}$$

- It's often tedious to keep track of two separate variables, both in terms of explanation and implementation. So, to avoid this situation, we can combine $\mathtt{W}$ and $\boldsymbol{b}$ together.
- To combine both the bias and weight matrix, we add an extra dimension (that is, column) to our input data $\mathtt{X}$ that holds a constant 1 – this is our bias dimension.

Gradient descent

The bias trick

- Typically we either append the new dimension to each individual x_i as the first dimension or the last dimension. In reality, it doesn't matter. We can choose any arbitrary location to insert a column of ones into our design matrix, as long as it exists.
- Doing so allows us to rewrite our scoring function via a single matrix multiply:

$$f(x_i, W) = Wx_i$$

- Again, we are allowed to omit the $\boldsymbol{b}$ term here as it is embedded into our weight matrix.

Gradient descent

The bias trick

- In the context of our previous examples in the "Animals" dataset, we've worked with $32 \times 32 \times 3$ images with a total of 3,072 pixels. Each x_i is represented by a vector of $[3072 \times 1]$.
  - Adding in a dimension with a constant value of one now expands the vector to be $[3073 \times 1]$.
  - Similarly, combining both the bias and weight matrix also expands our weight matrix W to be $[3 \times 3073]$ rather than $[3 \times 3072]$.
- In this way, we can treat the bias as a learnable parameter within the weight matrix that we don't have to explicitly keep track of in a separate variable.

Gradient descent

The bias trick

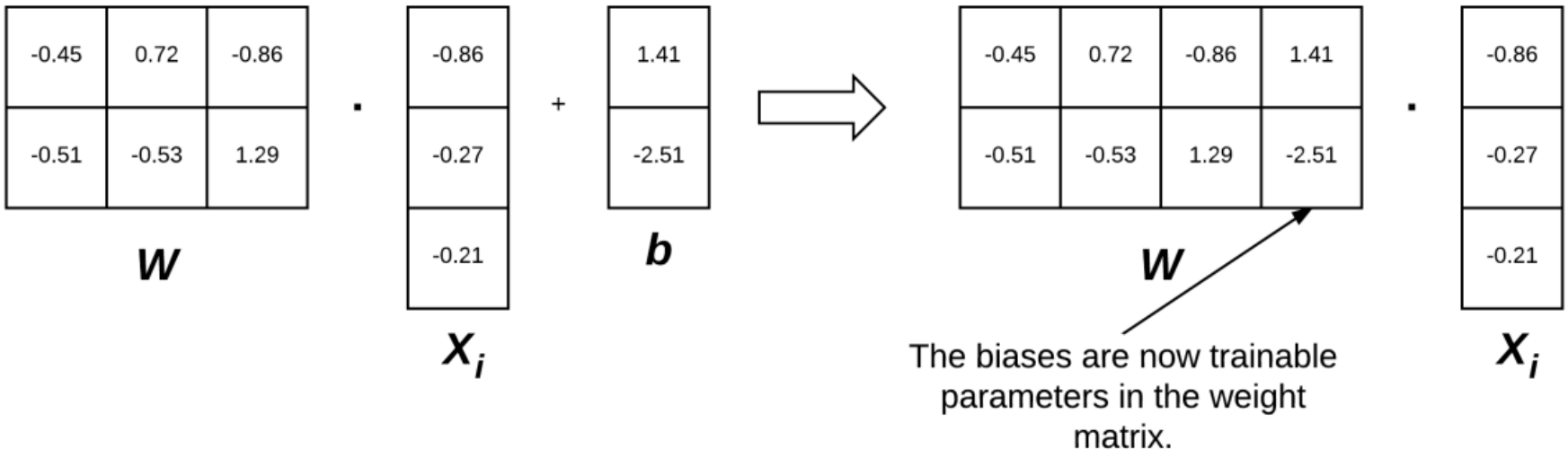

Figure: **Left**: Normally we treat the weight matrix and bias vector as two separate parameters. **Right:** However, we can actually embed the bias vector into the weight matrix (thereby making it a trainable parameter *directly inside* the weight matrix by initializing our weight matrix with an extra column of ones. Reprinted from

Contents

Gradient descent

Pseudocode for Gradient Descent

- Below is the Python-like pseudocode for the standard, vanilla gradient descent algorithm (pseudocode source: cs231n slides):

```
while True:
  Wgradient = evaluate_gradient(loss, data, W)
  W += -alpha * Wgradient
```

Gradient descent

Pseudocode for Gradient Descent

1. We start off on **Line 1** by looping until some condition is met, typically either:
   - A specified number of epochs has passed (meaning our learning algorithm has "seen" each of the training data points N times).
   - Our loss has become sufficiently low or training accuracy satisfactory high.
   - Loss has not improved in M subsequent epochs.

1. **Line 2** then calls a function named `evaluate_gradient`. This function requires three parameters:
   - loss: A function used to compute the loss over our current parameters W and input data.
   - data: Our training data where each training sample is represented by an image (or feature vector).
   - `W`: Our actual weight matrix that we are optimizing over. Our goal is to apply gradient descent to find a `W` that yields minimal loss.

- The `evaluate_gradient` function returns a vector that is K-dimensional, where K is the number of dimensions in our image/feature vector. The Wgradient variable is the actual gradient, where we have a gradient entry for each dimension.

1. We then apply gradient descent on **Line 3**. We multiply our Wgradient by alpha (α), which is our learning rate. The learning rate controls the size of our step.

Contents

Stochastic Gradient Descent (SGD)

- A first-order optimization algorithm that can be used to learn a set of classifier weights for parameterized learning. However, this vanilla implementation of gradient descent can be prohibitively slow to run on large datasets – in fact, it can even be considered computational wasteful.
- Stochastic Gradient Descent (SGD), a simple modification to the standard gradient descent algorithm that computes the gradient and updates the weight matrix `W` on small batches of training data, rather than the entire training set.

Stochastic Gradient Descent (SGD)

- While this modification leads to "more noisy" updates, it also allows us to take more steps along the gradient (one step per each batch versus one step per epoch), ultimately leading to faster convergence and no negative affects to loss and classification accuracy.
- SGD is arguably the most important algorithm when it comes to training deep neural networks. The engine that enables us to train large networks to learn patterns from data points.

Contents

Stochastic Gradient Descent (SGD)

Mini-batch SSD

- We need to compute prediction for each training point in our training data before we update the weight matrix makes the vanilla gradient descent slow. If we have over 1.2 million training images such as ImageNet, this computation can take a long time.
- It also turns out that computing predictions for every training point before taking a step along our weight matrix is computationally wasteful and does little to help our model coverage.

Stochastic Gradient Descent (SGD)

Mini-batch SSD

- Instead, what we should do is batch our updates. We can update the pseudocode to transform vanilla gradient descent to become SGD by adding an extra function call:

```
4        while True:
5            batch = next_training_batch(data, 256)
6            Wgradient = evaluate_gradient(loss, batch, W)
7            W += -alpha * Wgradient
```

Stochastic Gradient Descent (SGD)

Mini-batch SSD

- We added the `next_training_batch` function to the SGD. Instead of computing our gradient over the entire data set, we instead sample our data, yielding a batch.
- We evaluate the gradient on the batch, and update our weight matrix `W`.

Stochastic Gradient Descent (SGD)

Mini-batch SSD

- From an implementation perspective, we also try to randomize our training samples before applying SGD since the algorithm is sensitive to batches.
- In a "purist" implementation of SGD, your mini-batch size would be 1, implying that we would randomly sample one data point from the training set, compute the gradient, and update our parameters. However, we often use mini-batches that are > 1. Typical batch sizes include 32, 64, 128, and 256.

Stochastic Gradient Descent (SGD)

Mini-batch SSD

- So, why bother using batch sizes > 1? To start, batch sizes > 1 help reduce variance in the parameter update, leading to a more stable convergence. Secondly, powers of two are often desirable for batch sizes as they allow internal linear algebra optimization libraries to be more efficient.
- In general, the mini-batch size is not a hyperparameter we should worry too much about. We determine the training sample size to the nearest power of two that will fit on the GPU.

Contents

Extension to SGD

- There are two primary extensions that you'll encounter to SGD in practice.
  - The first is momentum, a method used to accelerate SGD, enabling it to learn faster by focusing on dimensions whose gradient point in the same direction.
  - The second method is Nesterov acceleration, an extension to standard momentum.

Contents

Extension to SGD

Momentum

- Consider your favorite childhood playground where you spent days rolling down a hill, covering yourself in grass and dirt. As you travel down the hill, you build up more and more momentum, which in turn carries you faster down the hill.
- Momentum applied to SGD has the same effect. Our goal is to build upon the standard weight update to include a momentum term, thereby allowing our model to obtain lower loss (and higher accuracy) in less epochs.
- The momentum term should, therefore, increase the strength of updates for dimensions who gradients point in the same direction and then decrease the strength of updates for dimensions who gradients switch directions.

Extension to SGD

Momentum

- Our previous weight update rule simply included the scaling the gradient by our learning rate:

$$\mathtt{W} = \mathtt{W} - \alpha \nabla_{\mathtt{W}} f(\mathtt{W})$$

- We now introduce the momentum term $\mathtt{V}$, scaled by γ:

$$\mathtt{V} = \gamma \mathtt{V} - \alpha \nabla_{\mathtt{W}} f(\mathtt{W}) \qquad \mathtt{W} = \mathtt{W} + \mathtt{V}$$

- The momentum term γ is commonly set to 0.9; although another common practice is to set γ to 0.5 until learning stabilizes and then increase it to 0.9. It is extremely rare to see momentum ¡ 0.5. For a more detailed review of momentum.

Contents

Extension to SGD

Nesterov's acceleration

- Let's suppose that you are back on your childhood playground, rolling down the hill. You've built up momentum and are moving quite fast but there's a problem. At the bottom of the hill is the brick wall of your school, one that you would like to avoid hitting at full speed.
- The same thought can be applied to SGD. If we build up too much momentum, we may overshoot a local minimum and keep on rolling. Therefore, it would be advantageous to have a smarter roll, one that knows when to slow down, which is where Nesterov accelerated gradient comes in.
- Nesterov acceleration can be conceptualized as a corrective update to the momentum which lets us obtain an approximate idea of where our parameters will be after the update. Looking at Hinton's Overview of mini-batch gradient descent slides, we can see a nice visualization of Nesterov acceleration.

Extension to SGD

Nesterov's acceleration

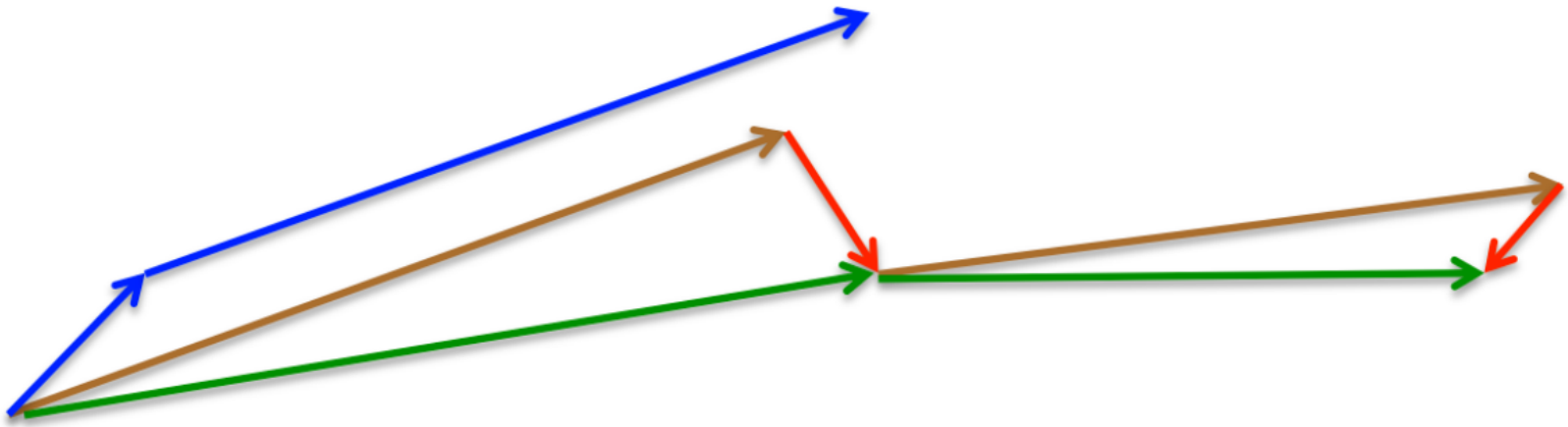

Figure: A graphical depecition of Nesterov acceleration. First, we make a big jump in the direction of the previous gradient, then measure the gradient where we ended up and make the correction.

Extension to SGD

Nesterov's acceleration

- Using standard momentum, we compute the gradient (small blue vector) and then take a big jump in the in the direction of the gradient (large blue vector).
- Under Nesterov acceleration we would first make a big jump in the direction of our previous gradient (brown vector), measure the gradient, and then make a correction (red vector) – the green vector is the final corrected update by Nesterov acceleration.

Contents

Extension to SGD

Anecdotal recommendations

- Momentum is an important term that can increase the convergence of our model; we tend not to worry with this hyperparameter as much, as compared to our learning rate and regularization penalty (coming up next), which are by far the most important knobs to tweak.
- Whenever using SGD, we recommend also apply momentum. In most cases, you can set it (and leave it) and 0.9, we can also start at 0.5 and increasing it to larger values as your epochs increase.

Extension to SGD

Anecdotal recommendations

- As for Nesterov acceleration, I tend to use it on smaller datasets, but for larger datasets (such as ImageNet), I almost always avoid it. While Nesterov acceleration has sound theoretical guarantees, all major publications trained on ImageNet (such as, AlexNet, VGGNet, ResNet, Inception, etc.) use SGD with momentum.
- Training deep networks on large datasets, SGD is easier to work with when using momentum and leaving out Nesterov acceleration. Smaller datasets, on the other hand, tend to enjoy the benefits of Nesterov acceleration. However, keep in mind that this is my anecdotal opinion and that your mileage may vary.

Contents

Regularization

"Many strategies used in machine learning are explicitly designed to reduce the test error, possibly at the expense of increased training error. These strategies are collectively known as regularization."

- We discussed two important loss functions: Multi-class SVM loss and cross-entropy loss. We then discussed gradient descent and how a network can actually learn by updating the weight parameters of a model.
- While our loss function allows us to determine how well our set of parameters are performing on a given classification task,

Regularization

- How do we go about choosing a set of parameters that help ensure our model generalizes well? Or, at the very least, lessen the effects of overfitting. The answer is regularization. Second only to your learning rate, regularization is the most important parameter of your model that can you tune.
- There are various types of regularization techniques, such as L1 regularization, L2 regularization (commonly called weight decay), and Elastic Net, that are used by updating the loss function itself, adding an additional parameter to constrain the capacity of the model.

Regularization

- We also have types of regularization that can be explicitly added to the network architecture. Dropout is the quintessential example of such regularization.
- We then have implicit forms of regularization that are applied during the training process. Examples of implicit regularization include data augmentation and early stopping.

Contents

Regularization

What is regularization and its need?

- Regularization helps us control our model capacity, ensuring that our models are better at making (correct) classifications on data points that they were not trained on, which we call the ability to generalize.
- If we don't apply regularization, our classifiers can easily become too complex and overfit to our training data, in which case we lose the ability to generalize to our testing data (and data points outside the testing set as well, such as new images in the wild).

Regularization

What is regularization and its need?

- However, too much regularization can be a bad thing. We can run the risk of underfitting, in which case our model performs poorly on the training data and is not able to model the relationship between the input data and output class labels (because we limited model capacity too much). For example, consider the following plot of points, along with various functions that fit to these points.

Regularization

What is regularization and its need?

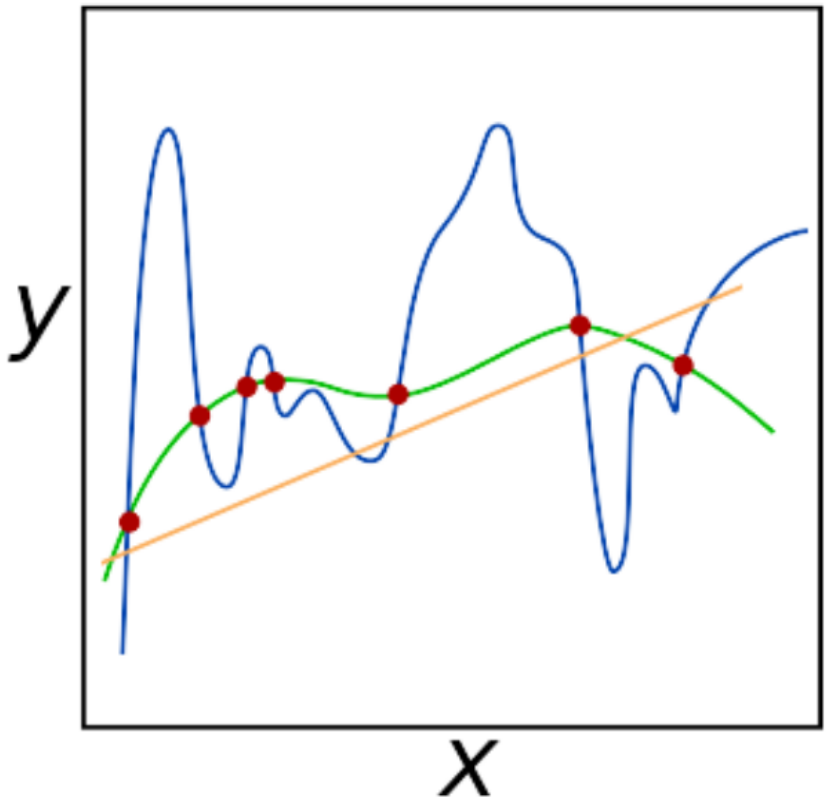

Figure: An example of underfitting (orange line), overfitting (blue line), and generalizing (green line). Our goal when building deep learning classifiers is to obtain these types of green functions that fit our training data nicely, but avoid overfitting. Regularization can help us obtain this type of desired fit.

Regularization

What is regularization and its need?

- The orange line is an example of underfitting. Unable to capture the relationship between the points. On the other hand, the blue line is an example of overfitting. Too many parameters in our model, and while it hits all points in the dataset, it also wildly varies between the points. It is not a smooth, simple fit that we would prefer.
- We then have the green function which also hits all points in our dataset, but does so in a much more predictable, simple manner.

Regularization

What is regularization and its need?

- The goal of regularization is to obtain these types of green functions that fit our training data nicely, but avoid overfitting to our training data (blue) or failing to model the underlying relationship (yellow).
- Regularization is a critical aspect of machine learning and we use regularization to control model generalization.

Contents

- Let's start with our cross-entropy loss function.

$$L_i = -\log(e^{S_{y_i}} / \sum_j e_j^s)$$

- The loss over the entire training set can be written as:

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i$$

- Now, let's say that we have obtained a weight matrix `W` such that every data point in our training set is classified correctly, which means that our loss $L = 0$ for all L_i.

Regularization

Updating Our Loss and Weight Update To Include Regularization

- Awesome, we are getting 100% accuracy but let me ask you a question about this weight matrix. Is it unique? Or, in other words, are there better choices of `W` that will improve our model's ability to generalize and reduce overfitting?
- If there is such a `W`, how do we know? And how can we incorporate this type of penalty into our loss function? The answer is to define a regularization penalty, a function that operates on our weight matrix. The regularization penalty is commonly written as a function, $R(\mathtt{W})$. The equation below shows the most common regularization penalty, L2 regularization (also called weight decay).

$$R(W) = \sum_i \sum_j W_{i,j}^2$$

Regularization

Updating Our Loss and Weight Update To Include Regularization

- What is the function doing exactly? In terms of Python code, it's simply taking the sum of squares over an array:

```
8                penalty = 0
9
10               for i in np.arange(0, W.shape[0]):
11                 for j in np.arange(0, W.shape[1]):
12                   penalty += (W[i][j] ** 2)
```

Regularization

Updating Our Loss and Weight Update To Include Regularization

- What we are doing here is looping over all entries in the matrix and taking the sum of squares. The sum of squares in the L2 regularization penalty discourages large weights in our matrix `W`, preferring smaller ones.
- Why might we want to discourage large weight values? In short, by penalizing large weights, we can improve the ability to generalize, and thereby reduce overfitting.
- The larger a weight value is, the more influence it has on the output prediction. Dimensions with larger weight values can almost single handedly control the output prediction of the classifier (provided the weight value is large enough, of course) which will almost certainly lead to overfitting.

Regularization

Updating Our Loss and Weight Update To Include Regularization

- To mitigate affect various dimensions have on our output classifications, we apply regularization, thereby seeking W values that take into account all of the dimensions rather than the few with large values.
- In practice you may find that regularization hurts your training accuracy slightly, but actually increases your testing accuracy.

Regularization

Updating Our Loss and Weight Update To Include Regularization

- Again, our loss function has the same basic form, only now we add in regularization:

$$L = \frac{1}{N} \sum_{i=1} NL_i + \lambda R(W)$$

- The first term we have seen before, it is the average loss over all samples in our training set.

Regularization

Updating Our Loss and Weight Update To Include Regularization

- The second term is new – this is our regularization penalty. The λ variable is a hyperparameter that controls the amount or strength of the regularization we are applying.
- In practice, both the learning rate α and the regularization term λ are the hyperparameters that you'll spending the most time tuning.

Regularization

Updating Our Loss and Weight Update To Include Regularization

- Expanding cross-entropy loss to include L2 regularization yields the following equation:

$$L = \frac{1}{N}\sum_{i=1}^{N}[-\log(e^{S_{y_i}} / \sum_{j} e^{S_{y_j}})] + \lambda \sum_{i}\sum_{j} \mathtt{W}_{i,j}^2$$

- We can also expand Multi-class SVM loss as well:

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}[max(0, s_j - s_{y_i} + 1)] + \lambda \sum_{i}\sum_{j} \mathtt{W}_{i,j}^2$$

Regularization

Updating Our Loss and Weight Update To Include Regularization

- Now, let's take a look at our standard weight update rule:

$$W = W - \alpha \nabla W f(W)$$

- This method updates our weights based on the gradient multiple by a learning rate α. Taking into account regularization, the weight update rule becomes:

$$W = W - \alpha \nabla W f(W) + \lambda R(W)$$

- Here we are adding a negative linear term to our gradients (i.e., gradient descent), penalizing large weights, with the end goal of making it easier for our model to generalize.

Contents

Regularization

Updating Our Loss and Weight Update To Include Regularization

- L2 regularization also called weight decay.
- L1 regularization
- Elastic Net regularization

The End